

Inference Optimization Profile

Design Partner Pilot · Pareton · xavier@pareton.ai

The fastest way to scope a pilot is this one-page profile. We've pre-filled every field with our best guess from public information — you just correct what's wrong. The two that matter most are your real serving config and your workload shape; the rest we can start from assumptions and refine together.

None of this requires your user data. A synthetic trace that matches the shape of your traffic is enough to run a meaningful pilot.

1. Target model

Field	Our best guess — please correct
Model + revision	GLM-5 — the exact checkpoint / weights version you serve in production
Quantization	FP8 (assumed). Correct if you serve FP16, AWQ, INT8, etc.

A patch tuned for FP8 is a different optimization than one for FP16, so the exact quant matters.

2. How you serve it today (the baseline)

Field	Our best guess — please correct
Serving stack + version	vLLM (recent release) — or your internal fork + commit
Tensor parallel	TP = 8 on a single node (assumed for GLM-5-scale on H200)
Key launch flags	max-num-seqs, max-num-batched-tokens, KV cache dtype, chunked prefill on/off, prefix caching on/off — your real values

This is the single most important section. Whatever config you give us becomes the baseline we optimize against, and we'll ask you to sign off that it's a fair one before we start. We want your real production config, not a default — a strawman baseline makes any result meaningless.

3. Workload shape

Field	Our best guess — please correct
Input length	p50 / p90 / p99 (assumed ~1K / 8K / 32K). Correct with your real distribution
Output length	p50 / p90 / p99 (assumed ~200 / 800 / 2K)
Shared prefix	How much of each request is shared — system prompts, few-shot templates, RAG context. (assumed moderate)
Concurrency / arrival	Sustained concurrent requests or requests/sec, and how bursty
Serving mode	Latency-serving (streaming/chat) vs throughput-serving (batch/offline)

Distributions, not just averages — a workload with mean input 1K but a 32K p99 stresses KV cache completely differently than a tight one. Shared-prefix ratio is often the single highest-leverage fact, so it's worth a real number if you have it.

4. Hardware

Field	Our best guess — please correct
Primary GPU type	H200 (assumed). Which SKU(s) do you actually run GLM-5 on?
Secondary GPU type(s)	e.g. a Blackwell SKU — ideally at least one more, so we can prove the gain holds across your fleet, not just one config

Cross-environment robustness is our core method: a gain that survives multiple GPU types is real; one that only works on a single config gets filtered out. Two GPU types makes for a far stronger pilot.

5. SLA bar

Field	Our best guess — please correct
Latency ceiling	p99 TTFT and p99 inter-token latency a change must stay under to count
Quality floor	What output quality / accuracy must be preserved (task eval, or output-equivalence tolerance)

This defines what “faster” is allowed to mean. Every optimization we report will have passed these gates — the cheapest way to fake a speedup is to silently degrade quality, and the SLA bar is what stops that.

6. Current baseline numbers

Field	Our best guess — please correct
Throughput today	Your measured tokens/sec or requests/sec at the above config
Latency today	Your measured p99 TTFT / inter-token latency

This is your “before.” It's also a fast sanity check — if our reproduced baseline doesn't match your real numbers, we want to find that out on day one, not at the end.

7. What a win looks like

Field	Our best guess — please correct
Priority metric	What you care about most: cost per token / throughput / TTFT / GPU utilization
Success threshold	The improvement that would make this worth a paid contract (e.g. $\geq X\%$ GPU-hour reduction at SLA)
Hard constraints	Anything a patch must NOT break — API compatibility, must stay on your fork, accuracy-sensitive paths, etc.

Agreeing the success threshold up front means a good pilot converts automatically instead of reopening negotiation at the end.

8. Logistics

Field	Our best guess — please correct
Technical point of contact	Who on your side owns the serving stack
Workload sharing	How you'd share a representative trace (real or synthetic) — under NDA if needed

How the pilot works

- You fill in / correct this profile. We agree on the baseline and the success threshold.
- We run our optimization process against your profile and benchmark every candidate across your GPU types under your SLA.
- You get a before/after report in GPU-hours saved — the number you'd actually bill on.
- If we hit the agreed threshold, we move to a savings-share contract. If not, you've spent a few weeks of low effort and lost nothing.

Questions or want to fill this in live on a call? — xavier@pareton.ai